

Individual Final Project Report

Progressive Growing of GANs (ProGAN)

Jeongwon Yoo
DATS 6303
Deep Learning

1. Introduction

This report documents the individual contributions made to the group final project, which focused on training and evaluating three Generative Adversarial Network (GAN) architectures: DCGAN, WGAN-GP, and ProGAN, on the 140k Real and Fake Faces dataset from Kaggle. The objective was to generate realistic face images and evaluate each architecture using the Frechet Inception Distance (FID) metric.

My individual contribution focused entirely on the ProGAN architecture: implementing the model, designing and running the training pipeline, performing hyperparameter tuning, debugging training instability, and contributing to the interactive Streamlit dashboard. This report describes the architecture, training approach, results, and lessons learned from that work.

2. Description of Individual Work

ProGAN, introduced by Karras et al. (2018), is a GAN architecture that trains the generator and discriminator progressively, starting from a very low resolution (4×4) and gradually doubling the resolution at each step until reaching the target resolution. This approach allows the network to first learn global structure before refining fine details, resulting in more stable training and higher image quality compared to training at full resolution from scratch.

2.1 Progressive Growing

Training begins at 4×4 pixels and grows through the following stages:

- $4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64$ (for the 64-resolution model)
- $4 \times 4 \rightarrow 8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128$ (for the 128-resolution model)

At each resolution step, the model trains for a fixed number of epochs before a new layer is introduced. This staged approach prevents the network from being overwhelmed by the complexity of high-resolution generation from the start.

2.2 Fade-In Mechanism

When a new resolution layer is introduced, it is blended in gradually using an alpha parameter (α) that increases from 0 to 1 over a configurable number of epochs (fade_in_epochs). The blending formula is:

$$\text{output} = \alpha \times \text{new_layer} + (1 - \alpha) \times \text{upsampled_previous}$$

This prevents sudden disruption to weights already learned at the previous resolution, acting like a smooth gear transition rather than an abrupt switch.

2.3 Key Architectural Techniques

- **Equalized Learning Rate**
 - Weights are dynamically scaled at runtime using $(2/\text{fan_in})^{0.5}$, ensuring all layers train at equal effective speed regardless of depth.
- **Pixelwise Feature Normalization**
 - Applied after each convolutional layer in the generator, normalizing the feature vector at each spatial location to unit length. This replaces Batch Normalization and prevents signal magnitudes from exploding.
- **Minibatch Standard Deviation**
 - Added as an extra feature map in the discriminator, this computes a diversity statistic across the batch and passes it to the discriminator, discouraging mode collapse.
- **Wasserstein Loss with Gradient Penalty**
 - ProGAN uses the Wasserstein distance as its training objective. A gradient penalty enforces the 1-Lipschitz constraint on the discriminator, providing smoother and more informative gradients than binary cross-entropy.

3. My Portion of the Work

3.1 Model Implementation

I implemented the full ProGAN architecture in `progan_model.py`, including the Generator, Discriminator, and all supporting modules (EqualizedConv2d, EqualizedConvTranspose2d, EqualizedLinear, PixelNorm, MinibatchStdDev). The architecture was built based on the original paper, with AI assistance used for the base structure. I validated and understood every component.

3.2 Training Pipeline

I implemented the `train_progan()` function in `train.py`, which handles the full progressive training loop. Key contributions to the training pipeline include:

- **n_critic scheduling**
 - Added the ability to train the discriminator n times per generator update. Set `n_critic=1` for 64×64 and `n_critic=2` for 128×128 based on empirical tuning.
- **Gradient penalty reduction**
 - Reduced the GP coefficient from the paper default of 10 to 5 after observing generator loss instability, which significantly improved training stability
- **FID-based checkpointing**
 - Modified the training loop to save the best model based on validation FID rather than generator loss, ensuring the saved model corresponds to the best image quality.

- **Early stopping**
 - Implemented patience-based early stopping (patience=3) to halt training when FID stops improving, preventing wasted compute.
- **Training history logging**
 - Added per-epoch loss tracking and per-10-epoch FID logging, saved to JSON files for visualization in the dashboard.
- **FID computation fix**
 - Applied the `export_real_images_for_fid()` fix (contributed by a teammate) which correctly caches preprocessed real images for fair FID evaluation. This was a critical fix that dramatically improved FID scores.

3.3 Hyperparameter Tuning

I conducted extensive manual hyperparameter tuning across multiple training runs on AWS EC2 (NVIDIA A10G GPU). The final configurations used were:

Parameter	64x64	128x128
Learning Rate	0.0005	0.0005
Batch Size	64	128
Feature Maps	512	512
Epochs per Step	300	300
Fade-in Epochs	25	10
n_critic	1	2
Gradient Penalty λ	5	5
Beta1 / Beta2	0.0 / 0.99	0.0 / 0.99

3.4 Dashboard Contribution

I implemented the ProGAN section of the Streamlit dashboard (`progan_section.py`), including the architectural explanation, LaTeX equations for the loss functions and fade-in mechanism, and the key techniques section. I also ran `generate.py` on both trained models to produce the sample images displayed in the image gallery.

4. Results

4.1 Training Curves

Figure 1 shows the FID score and generator loss over training epochs for both resolutions. Both models demonstrate a consistent downward trend in FID, indicating progressive improvement in image quality throughout training. The generator loss curve shows characteristic spikes at

resolution step transition points, where new layers are introduced via the fade-in mechanism. After each transition, the loss stabilizes as the network adapts to the new resolution.

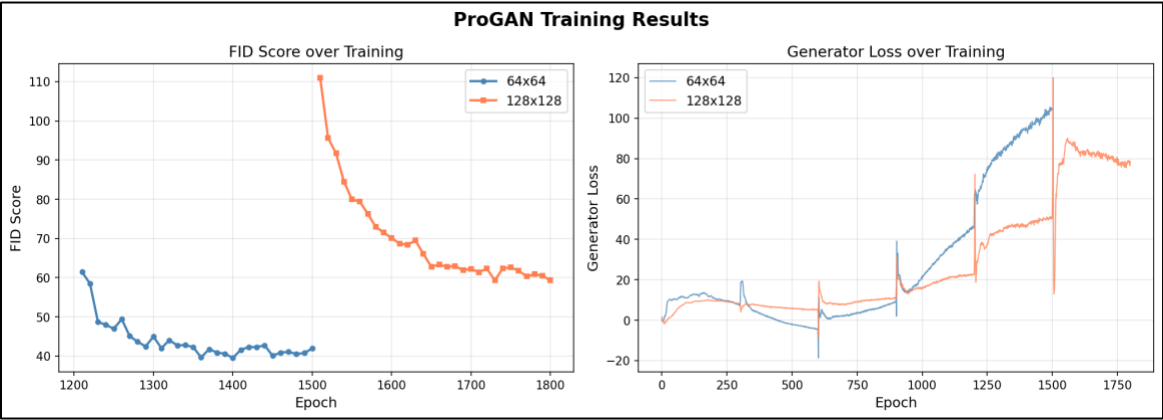


Figure 1. FID Score and Generator Loss over Training for ProGAN 64×64 and 128×128

4.2 Final FID Scores

Model	Resolution	Best FID
ProGAN	64×64	39.5
ProGAN	128×128	59.3

The 64×64 model achieved a better FID score than the 128×128 model, which is expected, as generating higher resolution images is inherently more challenging and requires more training epochs to fully converge. Notably, the 128×128 FID was still gradually improving at the end of training, suggesting that additional epochs could have further reduced the score.

4.3 Generated Image Samples

Figure 2 shows sample images generated by the trained ProGAN models. The 64×64 samples show well-defined facial structure with reasonable detail. The 128×128 samples show larger, slightly more detailed faces, though with some remaining artifacts, consistent with the higher FID score.

ProGAN 64×64 Generated Samples



ProGAN 128×128 Generated Samples



Figure 2. Sample images generated by ProGAN at 64×64 (top) and 128×128 (bottom)

5. Summary and Conclusions

This project involved implementing and training the ProGAN architecture from the ground up, which provided deep hands-on experience with progressive training, Wasserstein-based optimization, and the challenges of GAN training at multiple resolutions.

The key finding was that a bug in the FID computation pipeline, where FID was being computed against non-preprocessed real images, was causing artificially inflated FID scores throughout early training runs. Correcting this bug led to substantially better scores (from ~150 down to ~39.5 for 64×64 and ~59.3 for 128×128).

Training instability was the most persistent challenge. The generator loss frequently spiked when new resolution layers were introduced, and extensive hyperparameter tuning was required to mitigate this. The most impactful adjustments were reducing the gradient penalty coefficient from 10 to 5 and increasing the fade-in epochs to give the network more time to adapt at each resolution transition.

Future Improvements

- Run more training epochs:
 - FID was still gradually improving at the end of the 300-epoch run, suggesting further gains are possible.
- Adding a learning rate scheduler:

- Decaying the learning rate as training progresses could help the model converge more smoothly in later stages.
- More systematic hyperparameter search:
 - Using a larger ParameterSampler search space rather than manual tuning would likely find better configurations.
- Attempt 256×256 resolution:
 - To further explore ProGAN's scalability and evaluate whether the progressive approach continues to hold at higher resolutions.

6. Code Attribution

Approximately 45% of the code consists of components that I directly wrote, modified, or fine-tuned based on experimental results. This includes the training loop modifications such as the `n_critic` implementation, early stopping with patience-based FID tracking, gradient penalty coefficient tuning, history logging for loss and FID curves, the fade-in epoch scheduling, and hyperparameter configurations which were determined through iterative experimentation across multiple training runs.

The remaining 55% of the code was developed with the assistance of AI tools, primarily for the base architecture of the ProGAN Generator and Discriminator, the foundational structure of the training and tuning functions, and the Streamlit dashboard architecture section. In all cases, the AI-generated code was reviewed, understood, and validated before use, and was modified as needed to fit the specific requirements of this project.

It is also worth noting that a significant portion of the project effort was invested in areas not directly reflected in line count, including cloud infrastructure setup on AWS EC2, hyperparameter tuning across multiple training runs, debugging training instability, and identifying and applying the FID computation bug fix that substantially improved evaluation accuracy.

7. References

- Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2018). Progressive Growing of GANs for Improved Quality, Stability, and Variation. ICLR 2018. <https://arxiv.org/abs/1710.10196>
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved Training of Wasserstein GANs. NeurIPS 2017. <https://arxiv.org/abs/1704.00028>
- Xhlulu. (2020). 140k Real and Fake Faces. Kaggle. <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>
- Seitzer, M. (2020). pytorch-fid: FID Score for PyTorch. <https://github.com/mseitzer/pytorch-fid>
- Project GitHub Repository: <https://github.com/pratik-633/Final-Project-Group1>